

ReelsFlow - Documentação de Arquitetura de Backend e Banco de Dados v1.0

Este documento detalha rigorosamente a infraestrutura de backend, a modelagem de dados no Supabase (PostgreSQL), os contratos de rotas utilizando Next.js API Handlers e os mecanismos de sincronização assíncrona. Ele foi arquitetado para servir como engenharia de contexto avançada para geradores de código e arquitetos de IA.

1. Arquitetura do Backend e Stack Tecnológica

O backend do ReelsFlow segue um modelo híbrido focado em microsserviços orientados a eventos, dividindo responsabilidades entre manipulação de requisições síncronas e orquestração de mídia computacionalmente pesada.

- **Camada de Aplicação (Core API):** Next.js App Router (Route Handlers executados em Node.js no ambiente servless ou edge). Funciona como proxy seguro de controle de acesso, gerencia a lógica transacional do Stripe e serve como barreira de segurança antes de disparar automações dispendiosas.
- **Camada de Persistência (Database):** Supabase rodando uma instância gerenciada de PostgreSQL. Além do armazenamento relacional tradicional, o Supabase fornece autenticação integrada (Supabase Auth) e Row Level Security (RLS) nativo.
- **Orquestração de Processos Assíncronos:** n8n rodando em container Docker autônomo, acionado por webhooks criptografados do Next.js para gerenciar pipelines de inteligência artificial de longa duração.
- **Armazenamento de Objetos (Storage):** Cloudflare R2 ou Supabase Storage para persistência estável dos arquivos binários finais (.mp4 e .mp3) com distribuição CDN global para atendimento dos requisitos de leitura da Meta Graph API.

2. Modelagem do Banco de Dados (Supabase / PostgreSQL)

Abaixo estão especificados os esquemas relacionais exatos das tabelas do sistema. Todas as tabelas operam sob a timezone UTC padrão.

2.1. Tabela: users

Armazena o perfil do usuário na aplicação, estendendo o esquema nativo do auth.users do Supabase.

Coluna	Tipo de Dados	Restrições / Chaves	Descrição
id	UUID	PRIMARY KEY, REFERENCES auth.users(id)	Identificador exclusivo espelhado do módulo de autenticação.
email	TEXT	UNIQUE, NOT NULL	Endereço de email corporativo do cliente.
credits_balance	INTEGER	NOT NULL, DEFAULT 0, CHECK (credits_balance >= 0)	Saldo ativo de créditos disponíveis para geração de Reels.
stripe_customer_id	TEXT	UNIQUE	Mapeamento do identificador do cliente dentro do gateway Stripe.
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL, DEFAULT now()	Data e hora de criação da conta.

2.2. Tabela: instagram_credentials

Guarda tokens de longa duração e metadados das contas comerciais integradas do Instagram.

Coluna	Tipo de Dados	Restrições / Chaves	Descrição
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Identificador único da credencial vinculada.
user_id	UUID	NOT NULL, REFERENCES users(id) ON DELETE CASCADE	Chave estrangeira apontando para o dono da credencial.

Coluna	Tipo de Dados	Restrições / Chaves	Descrição
instagram_business_id	TEXT	NOT NULL	ID único da conta do Instagram Business fornecido pela Meta.
encrypted_access_token	TEXT	NOT NULL	Token criptografado via AES-256 no nível da aplicação.
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL, DEFAULT now()	Última atualização do token (essencial para controle de expiração).

2.3. Tabela: video_jobs

Rastreia os estados de processamento computacional de cada requisição de vídeo iniciada.

Coluna	Tipo de Dados	Restrições / Chaves	Descrição
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	ID único do job de vídeo.
user_id	UUID	NOT NULL, REFERENCES users(id)	Dono solicitante da renderização de vídeo.
status	TEXT	NOT NULL, CHECK (status IN ('pending', 'scripting', 'rendering', 'ready', 'failed'))	Estado atual da máquina de estados do n8n.
prompt_input	TEXT	NOT NULL	Texto bruto inserido pelo usuário na UI.
script_json	JSONB	DEFAULT '{}::jsonb	Roteiro estruturado retornado pelo LLM com timestamps.

Coluna	Tipo de Dados	Restrições / Chaves	Descrição
video_url	TEXT	-	Link público final do asset .mp4 hospedado no R2/S3.

2.4. Tabela: **scheduled_posts**

Controla as postagens configuradas e filas de envio cronometradas para a API do Instagram.

Coluna	Tipo de Dados	Restrições / Chaves	Descrição
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Identificador global do post agendado.
video_job_id	UUID	NOT NULL, REFERENCES video_jobs(id) ON DELETE CASCADE	Referência cruzada para extração da URL mestre do mp4.
caption	TEXT	NOT NULL	Legenda textual, incluindo hashtags geradas para SEO do Instagram.
scheduled_for	TIMESTAMP WITH TIME ZONE	NOT NULL	Timestamp do gatilho cronológico de publicação.
post_status	TEXT	NOT NULL, CHECK (post_status IN ('scheduled', 'publishing', 'published', 'failed'))	Status do processamento de entrega na Meta Graph API.

3. Engenharia de Rotas e Webhooks (Next.js API

Routes / n8n)

A comunicação entre o Next.js e o n8n deve ser protegida por assinatura de token em cabeçalho HTTP para evitar injeções maliciosas na fila de renderização.

- **POST /api/video/generate:** Invocada pelo cliente frontend. Esta rota executa um bloco transacional no Supabase (RPC) para validar e debitar 1 crédito do usuário. Caso o balanço seja positivo, encaminha o payload limpo contendo `job_id` e `prompt_input` para o webhook seguro de processamento pesado do n8n através de chamada mútua assinada via chave secreta `X-ReelsFlow-Signature`.
- **POST /api/webhooks/n8n-callback:** Rota interna de escuta do Next.js exposta exclusivamente para o n8n atualizar o status do processamento. O n8n envia atualizações conforme progride (ex: status: 'rendering') e, ao finalizar, injeta a `video_url` pública gerada e muda o status para 'ready', disparando um evento de broadcast reativo para a interface web.

4. Políticas de Segurança de Linha (RLS - Row Level Security)

Para garantir isolamento completo e conformidade de segurança e privacidade de dados, nenhuma consulta direta de tabelas é exposta publicamente sem a validação do contexto de sessão do Supabase JWT (`auth.uid()`).

```
-- Habilitando RLS globalmente nas entidades críticas
ALTER TABLE public.users ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.video_jobs ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.scheduled_posts ENABLE ROW LEVEL SECURITY;

-- Política de Leitura/Escrita para Jobs de Vídeo
CREATE POLICY "Usuários podem interagir apenas com seus próprios jobs"
ON public.video_jobs
FOR ALL
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);
```

5. Fluxos Críticos do Ciclo de Dados

Durante a criação de conteúdo, o tratamento transacional do banco de dados deve seguir uma política rígida contra falhas (Rollback Policy). Se a chamada para o motor de IA ou renderização falhar em qualquer nó intermediário do n8n, o n8n deve disparar um webhook de contingência que executa a função PostgreSQL nativa `public.refund_video_credit(user_id)`,

reestabelecendo de forma imediata o saldo de créditos do cliente e registrando o erro na tabela de log correspondente.